

ПОДГОТОВКА ЗА ДЗИ ПО ИНФОРМАТИКА

Синтезирана теория: Основи на C# и анализ на програмен код

Учебен материал за самостоятелна подготовка преди Среща 2

Целта на този материал е да възстанови най-важните знания по C#, които са необходими за решаване на задачи от държавния зрелостен изпит. След всеки раздел изпълнявайте примерите и се опитвайте да обясните какво прави кодът.

1. Променливи и типове данни

Променливата е именувана област от паметта, в която се съхранява стойност.

```
int age = 18;
double averageGrade = 5.25;
char symbol = 'A';
string name = "Maria";
bool isPassed = true;
```

Тип	Предназначение	Пример
int	Цели числа	25
double	Дробни числа	5.75
decimal	Точни финансови стойности	12.50m
char	Един символ	'A'
string	Текст	"Hello"
bool	Логическа стойност	true / false

Важно: Когато делим две стойности от тип `int`, резултатът също е цяло число.

```
int a = 5;
int b = 2;
Console.WriteLine(a / b);    // 2

double result = (double)a / b;
Console.WriteLine(result);   // 2.5
```

2. Вход и изход

За извеждане използваме `Console.WriteLine()`, а за въвеждане - `Console.ReadLine()`.

```
Console.WriteLine("Hello");

string name = Console.ReadLine();
int number = int.Parse(Console.ReadLine());
```

По-безопасно преобразуване:

```
bool isValid = int.TryParse(Console.ReadLine(), out int number);
```

3. Аритметични оператори

Оператор	Действие
+	Събиране
-	Изваждане
*	Умножение
/	Деление
%	Остатък при деление

Последна цифра и премахване на последната цифра:

```
int number = 527;
int lastDigit = number % 10; // 7
```

```
number = number / 10;          // 52
```

4. Оператори за сравнение и логически оператори

Операторите за сравнение връщат стойност true или false.

```
number == 5    // равно
number != 5    // различно
number > 5
number < 5
number >= 5
number <= 5
```

Важно: Операторът = присвоява стойност, а == проверява равенство.

```
if (age >= 18 && age <= 65)
{
    Console.WriteLine("Допустима възраст");
}

if (day == "Saturday" || day == "Sunday")
{
    Console.WriteLine("Почивен ден");
}
```

5. Условни конструкции

```
if (condition)
{
    // изпълнява се при вярно условие
}
else
{
    // изпълнява се при невярно условие
}
```

Пример за проверка дали число е четно:

```
int number = 8;

if (number % 2 == 0)
{
    Console.WriteLine("Even");
}
else
{
    Console.WriteLine("Odd");
}
```

При else if условията се проверяват отгоре надолу. Изпълнява се първият блок с вярно условие.

```

if (grade >= 5.50)
{
    Console.WriteLine("Excellent");
}
else if (grade >= 4.50)
{
    Console.WriteLine("Very good");
}
else
{
    Console.WriteLine("Other");
}

```

6. Цикъл for

Използва се, когато знаем или можем да определим броя на повторенията.

```

for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}

```

Цикълът извежда числата 0, 1, 2, 3 и 4.

Пример за намиране на сума:

```

int sum = 0;

for (int i = 1; i <= 5; i++)
{
    sum += i;
}

Console.WriteLine(sum); // 15

```

7. Цикъл while

Използва се, когато повторението зависи от условие.

```

int number = 527;
int sum = 0;

while (number > 0)
{
    int digit = number % 10;
    sum += digit;
    number /= 10;
}

Console.WriteLine(sum); // 14

```

number	digit	sum
527	7	7

52	2	9
5	5	14
0	-	14

8. Масиви

Масивът съхранява множество стойности от един и същ тип.

```
int[] numbers = { 4, 7, 2, 9, 6 };
```

Индекс	Стойност
0	4
1	7
2	2
3	9
4	6

```
Console.WriteLine(numbers[0]);           // първи елемент
Console.WriteLine(numbers.Length);        // брой елементи
Console.WriteLine(numbers[numbers.Length-1]); // последен елемент

for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i]);
}
```

9. Основни алгоритми с масиви

9.1. Сума

```
int sum = 0;

for (int i = 0; i < numbers.Length; i++)
{
    sum += numbers[i];
}
```

9.2. Преброяване по условие

```
int count = 0;

for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] < 0)
    {
        count++;
    }
}
```

9.3. Максимален елемент

```
int max = numbers[0];

for (int i = 1; i < numbers.Length; i++)
{
    if (numbers[i] > max)
    {
        max = numbers[i];
    }
}
```

Важно: Не задавайте винаги `max = 0`. Това е грешно, ако всички числа са отрицателни.

9.4. Минимален елемент

```
int min = numbers[0];

for (int i = 1; i < numbers.Length; i++)
{
    if (numbers[i] < min)
    {
        min = numbers[i];
    }
}
```

10. Методи

Методът е именуван блок от код, който изпълнява определена задача.

```
static int Sum(int a, int b)
{
    return a + b;
}

int result = Sum(3, 5);
```

Метод без върната стойност използва `void`:

```
static void PrintMessage(string message)
{
    Console.WriteLine(message);
}

PrintMessage("Hello");
```

11. Типични грешки

11.1. Грешна граница на масив

```
// Грешно
for (int i = 0; i <= numbers.Length; i++)
```

```
// Правилно
for (int i = 0; i < numbers.Length; i++)
```

11.2. Безкраен цикъл

```
int i = 0;

while (i < 10)
{
    Console.WriteLine(i);
    i++; // задължителна промяна
}
```

11.3. Целочислено деление

```
double wrong = 5 / 2;      // 2.0
double correct = 5.0 / 2; // 2.5
```

11.4. Неинициализирана променлива

```
// Грешно
int sum;
sum += 5;

// Правилно
int sum = 0;
sum += 5;
```

12. Контролни въпроси

1. Каква е разликата между int и double?
2. Какъв тип стойност връща Console.ReadLine()?
3. Каква е разликата между = и ==?
4. Какво намира операторът %?
5. От кой индекс започва масивът?
6. Как се намира индексът на последния елемент?
7. Кога използваме for и кога while?
8. Какво прави return?
9. Каква е разликата между && и ||?
10. Защо max = 0 може да даде грешен резултат?

13. Кратки задачи за самопроверка

1. Какво ще бъде изведено?

```
int sum = 0;

for (int i = 2; i <= 8; i += 2)
{
    sum += i;
}
```

```
}  
  
Console.WriteLine(sum);
```

2. Какво ще бъде изведено?

```
int number = 342;  
int count = 0;  
  
while (number > 0)  
{  
    number /= 10;  
    count++;  
}  
  
Console.WriteLine(count);
```

3. Открийте и поправете грешката:

```
int[] numbers = { 2, 4, 6 };  
  
for (int i = 1; i <= numbers.Length; i++)  
{  
    Console.WriteLine(numbers[i]);  
}
```

4. Напишете програма, която намира произведението на цифрите на положително цяло число.

5. Създайте метод CountEven(int[] numbers), който връща броя на четните елементи.

14. Какво трябва да можете след този материал

- да разпознавате основните типове данни;
- да използвате вход и изход;
- да работите с условни конструкции;
- да проследявате цикли for и while;
- да обхождате масив без грешка в индексите;
- да намирате сума, брой, минимум и максимум;
- да разбирате и създавате прости методи;
- да откривате типични синтактични и логически грешки.